Synopsis

The Lessons of ValuJet 592 was written by pilot William Langewiesche, who draws on his experience and journalistic expertise to recount the infamous crash of a ValuJet DC-9 in May 1996. As a magazine article, it starts in the middle of the story, leading with the most emotional and intense moment: the crash. The more important part of the story, especially when considering the factors and causes of the tragedy, and the *lessons* we can learn from it, is found in the events leading up to the flight.

The immediate cause of the crash of the ValuJet craft was the oxygen generators on board, which, when activated, could heat up to 500 degrees Fahrenheit, and if near other generators, could heat up significantly hotter than that. At this temperature, many substances will ignite, and the rubber tires also being transported in the cargo hold below and behind the cockpit did so. From there, the fire spread to the rest of the aircraft. The plane eventually lost power over the Everglades and fell into the Everglades Holiday Park, its hull shattering as it landed, the impact killing all 110 people on board.

The reason that these oxygen generators turned on, or that they were even on the aircraft in the first place, was due to a myriad of systemic failures in the combined systems of ValuJet and SabreTech, in addition to the actual human members of this system. The issue started with the mechanics at SabreTech packing away these generators to be shipped; they ignored the work order to install safety caps on the generators (which would have prevented them from activating). They did this not only because they failed to put safety measures first, but also because they were not given any safety caps with which to properly secure the generators. This discrepancy passed through a number of hands and saw numerous negligent acts by employees at SabretTech and ValuJet, causing the information as to what the items exactly were (oxygen containers vs. *hazardous* oxygen generators) to be lost in translation.

The events summarized here are an example of what Langeweische calls a *system accident*, or a *normal accident*, which comes from a theory created by Charles Perrow, a sociologist, whose research led him to believe that any large and sufficiently complex organization may reach a point where failure generated by the natural slight deviations in one tiny subsystem is “a virtual guarantee”- a result of “interactive complexity” and “tight coupling”. The former is a term that describes many systems or complex systems or many complex systems, etc., which work together in a complex way; the latter means specifically that elements of these systems are *linked* in multiple ways. The result of this is that a failure in one part can cause another failure, which can cause another failure, and so on, until catastrophic failure occurs. Langeweische, while being a pilot himself and having come from an environment where a can-do attitude is prized, did finally come around to the idea of unpreventable failure. The article concludes with the idea that increasing a system's complexity to prevent failure of the procedural or engineered variety (these are simple issues in specific self-contained systems) is the very thing that can lead to a higher rate of failure.

In the Context of Software Development

The idea of a system accident easily connects to the realm of software development. While Langeweische makes the distinction between engineered accidents and system accidents, it just so happens that software is usually constructed of complex systems. While these systems are not as unpredictable as those directly involving humans in their functioning, humans are inherent in the creation of software systems. If we add the extra layer of construction, maintenance, and, of course, the *user* of a complex software system, we begin to see something that has key similarities to the kind of complex system illustrated in the article.

Software is not exactly the same as the kind of system in the article, however, because software inherently lets us bind its behavior much more strictly than a system executed by humans. One example of this is that error handling can be automated in predictable ways. The smaller amount of human decision-making on the execution side, and the ability to gate human intents/actions, allows us to much more *reliably* abstract the many possible errors a complex software system can have, which in turn allows us to manage them smoothly.

Where normal accidents can still creep in is through system vulnerability to outside influence and our increasing use of AI. Along with the element of human error that software still has (at least for now), the existence of “bad actors” over the internet adds a whole other realm of possibility for system failure, except now that failure has an actual agent behind it, not the “mind of its own” that Langeweische ascribes to his type of system failure. The implication of this is that as we improve our software, we also improve the software with which a hacker, for example, might steal data, or money, or hijack a system, etc. With the advent of AI, we have software writing software, or maybe intelligence writing software- intelligence that we allow to access a huge amount of compute power and storage. While many might use AI to shore up their software’s security and find and/or patch vulnerabilities, it also means that we are having a complex system- the software- interact with another, not only a complex system, but an inherently *probabilistic* system, making the combined ecosystem many times more complex.

Another way for normal accidents to happen is through poor communication- this *does* have a 1-to-1 relationship within the article. The jargon of the engineers and lack of communication, and specifically *verification* of understanding (which is itself a subset of “communication”) by the managers and policy-makers at ValuJet and SabreTech, are what caused the initial failure of the mechanics to understand the danger associated with the oxygen containers- of course they were negligent in not putting the safety caps on them, but this was also the fault of poor communication- the different departments of ValuJet and SabreTech should have communicated better to make sure all required materials for an operation were available. In software development, poor communication results in deliverables not meeting the required specifications, or possibly that specifications/requirements/needs changed, and the final output now does not meet those specifications. This can result in partial or total system failure if the

deployed software is a subsystem of something larger, or hopefully something less catastrophic, like a missed deadline. In both cases, a missed requirement or forgotten update message can derail the integrity of a software development lifecycle. In the case of ValuJet, it was specifically the lack of *checking* with other people that caused the initial mix-up and negligence. The article goes on to say that those mechanics should not have been expected to do the large amount of research on their own to figure out what exactly to do, and on a moral level this is debatable, but from an operational standpoint this is completely true, because if they were expected to do that it would mean that their execution of their work would take 10 times as long- communication while solving potential errors down the line can also increase efficiency of development.